

3.3 & 3.4 File Handling and Operations (Modes r, w, a)

فائل ہینڈلنگ سے ڈیٹا ڈسک پر مستقل (Persistent) رہتا ہے، برخلاف RAM کے متغیرات کے جو پروگرام بند ہوتے ہی مٹ جاتے ہیں۔

CS XII

Chapter 3: Programming Fundamentals

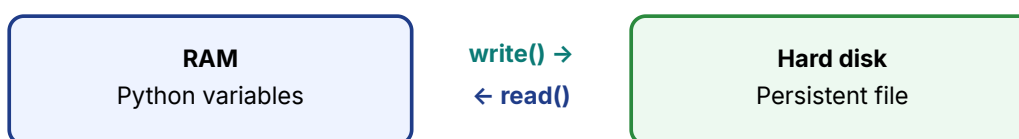
Lecture 14 of 18

★ Golden

3.3 & 3.4 File Handling and Operations (Modes r, w, a) ★ Golden

LEARN IT

File handling is a program's ability to create, read, write, update and delete files on disk. Variables live in **RAM** and vanish when the program ends. Files live on a **hard disk or SSD** and stay there — they are **persistent**.



Read copies data from disk into RAM. Write copies data from RAM onto disk.

Types of files

Type	What it stores	Examples
Text file	Human-readable characters. You can open it in a text editor.	.txt, .csv, .py
Binary file	Raw bytes, not meant to be read as text.	.jpg, .mp3, .dat

This chapter uses text files.

The `open()` function and modes

You must open a file before you use it: `open(filepath, mode)`.

Mode	Name	Description
'r'	Read	Open for reading. Fails if the file does not exist.
'w'	Write	Open for writing. Erases existing content. Creates the file if it is missing.
'a'	Append	Open for writing at the end . Keeps old content. Creates the file if it is missing.
'rb' / 'wb'	Binary	Read or write bytes (images, audio).

File object methods

Method	What it does
<code>.read()</code>	Whole file as one string.
<code>.readlines()</code>	All lines as a list of strings (each usually ends with <code>\n</code>).
<code>.write("text")</code>	Writes a string. You must add <code>\n</code> yourself for a new line.
<code>.close()</code>	Saves to disk, closes the file, frees RAM. Essential if you did not use <code>with</code> .

WORKED EXAMPLE

Create a file in write mode.

```
new_file = open('myfile.txt', 'w')
new_file.write("The quick brown fox jumps over the lazy dog!!!\n")
new_file.close()
print("File has been created successfully.")
```

OUTPUT

```
File has been created successfully.
```

If `myfile.txt` did not exist, `'w'` created it. The sentence is now on disk. `.close()` flushes the buffer so the bytes actually reach the disk.

Danger of `'w'`: opening an existing file in `'w'` deletes everything first.

```
f = open('myfile.txt', 'w')
f.write("FIRST\n")
f.close()
f = open('myfile.txt', 'w')
f.write("SECOND\n")
f.close()
with open('myfile.txt', 'r') as f:
    print(f.read(), end="")
```

OUTPUT

```
SECOND
```

`FIRST` is gone. To keep old lines and add a new one, use `'a'`:

```
f = open('notes.txt', 'w')
f.write("Line 1\n")
f.close()
f = open('notes.txt', 'a')
f.write("Line 2\n")
f.close()
with open('notes.txt', 'r') as f:
    print(f.read(), end="")
```

OUTPUT

```
Line 1
Line 2
```

REMEMBER IT

r = Read existing, w = Wipe then write, a = Add at end. Remember: `write` does not add a newline unless you type `\n`.

COMMON MISTAKE

Forgetting `.close()` can leave data in a memory buffer so the file on disk looks empty or incomplete — or the file can stay locked. Section 3.6 shows `with`, which closes for you even after an error. Never use `'w'` when you only meant to add a line; that is the most common way students wipe their own data.

اردو میں سمجھیں

فائل ہینڈلنگ سے ڈیٹا ڈسک پر مستقل (Persistent) رہتا ہے، برخلاف RAM کے متغیرات کے جو پروگرام بند ہوتے ہی مٹ جاتے ہیں۔ `open(path, mode)` سے فائل کھلتی ہے: `'r'` پڑھنے کے لیے (فائل ہونی چاہیے)، `'w'` لکھنے کے لیے (پرانا مواد مٹ جاتا ہے)، `'a'` آخر میں جوڑنے کے لیے۔ کام کے بعد `.close()` ضروری ہے اگر `with` استعمال نہ ہو۔

EXAM TIP

Define file handling, contrast RAM with disk, list modes `r / w / a` with one property each, and warn that `w` overwrites. Short programs that write one line and close the file are common. Mention `.read()`, `.readlines()`, `.write()` and `.close()` by name.

CHECK YOURSELF

1. Which mode overwrites an existing file?
2. Which mode fails if the file is missing?
3. Why must `.write` often include `\n`?

Answers: 1. `'w'`. 2. `'r'`. 3. Because `write()` does not start a new line by itself; without `\n` the next write continues on the same line.

Next lecture: 3.6 The `with` Statement (Context Manager)